

Reineke-RAG

Technische Beschreibung

Reineke-RAG — 2026

Reineke-RAG — Technische Beschreibung

Eine Experten-Beschreibung dessen, was dieses Repository heute enthält, was es einmal werden soll und wie beide Zustände zusammenhängen.

1. Was dieses Repository heute ist

Reineke-RAG ist — **zum jetzigen Zeitpunkt seines Lebenszyklus** — ein **Blaupausen-Repository**: Konzept, Architektur, Architecture Decision Records (ADRs), phasenorientierter Implementierungsplan, Subagent-Briefings sowie ein Skelett aus Docker Compose, Makefile und `.env`-Vorlage.

Anwendungscode ist noch nicht enthalten. Der eigentliche Aufbau wird durch einen KI-Koordinator-Agenten erzeugt, der Arbeitspakete an sieben spezialisierte Sub-Agenten delegiert (definiert in [docs/06_AGENT_BRIEFS.md](#)).

Das Repository steht auf **zwei Verträgen**:

1. **Konzept + ADRs sind unveränderlich**, solange kein neues ADR ein altes ersetzt. Siehe [CLAUDE.md](#) § „House rules“.
2. **Verantwortungs-Grenzen** zwischen den Sub-Agenten sind strikt; jede Datei hat genau einen Eigentümer.

Das macht den Build reproduzierbar: jeder neu gestartete Koordinator findet denselben Plan, dieselben Abnahmekriterien, dieselben Übergabepunkte vor.

1.1 Repository-Struktur

/	
├─ README.md	Orientierung
├─ CLAUDE.md	Projekt-Anweisungen für KI-Agenten
├─ Makefile	Top-Level-Ziele (bootstrap, up, pull-models...)
├─ BUILD_LOG.md	reine Protokoll-Datei aller Build-Ereignisse (vom Koo
├─ HANDOVER.md	Abnahme-Template; wird in Phase 10 ausgefüllt
├─ .gitignore	
├─ .claude/	
│ └─ settings.local.json	
├─ agents/	acht Sub-Agent-Definitionen (.md mit Frontmatter)
├─ agents/	(leer; reserviert)
├─ config/	
│ └─ .env.example	vollständige Environment-Vorlage
│ └─ docker-compose.yml	Compose-Skelett (25 Dienste, gepinnte Tags, 2 Profile
│ └─ owner-inputs.yaml.example	Eigentümer-seitige Phase-0-Eingaben
├─ docs/	
│ └─ 01_CONCEPT.md	Nordstern-Konzept, Anforderungen, Technologieentschei
│ └─ 02_ARCHITECTURE.md	Komponenten, Datenflüsse, Schemata, Schnittstellen
│ └─ 03_HANDBOOK.md	Endnutzer-Handbuch (Zielverhalten)
│ └─ 04_OPERATIONS.md	Admin-/IT-Handbuch (Zielverhalten)
│ └─ 05_IMPLEMENTATION_PLAN.md	phasenorientierter Bauplan mit Abnahmekriterien
│ └─ 06_AGENT_BRIEFS.md	Koordinator + 7 spezialisierte Sub-Agent-Spezifikatio
└─ adr/	acht akzeptierte Architecture Decision Records

1.2 Was eine Leserin / ein Leser aktuell tun kann

- Das Konzept und die Architektur vollständig durchlesen.
 - Jedes gepinnte Container-Image und dessen Rolle prüfen.
 - Den Build reproduzieren, indem der `coordinator`-Subagent mit ausgefüllter `config/owner-inputs.yaml` gestartet wird.
 - Die Entscheidungs-Begründungen für jede relevante Wahl nachlesen (acht ADRs).
 - **Nicht** `make up` ausführen — der konkrete Service-Code (`services/**`) entsteht erst während des Builds und ist nicht vorab eingesehen.
-

2. Das Zielsystem

Fertig gebaut (Phase 10 mit Unterschrift des Eigentümers) ist Reineke-RAG ein **voll-offline, enterprise-taugliches Retrieval-Augmented-Generation-System (RAG)** für interne Word-, PDF- und Excel-Dokumente in einem gemischten deutsch/englischen Korpus. Einzelhost-Deployment über Docker Compose, ausgelegt für eine Apple-M4-Max-64-GB-Referenzmaschine oder Linux mit ≥ 24 GB GPU.

2.1 Kernfähigkeiten

1. **Ingest von PDF, DOCX, XLSX** — einschließlich gescannter PDFs (OCR), mehrspaltiger Layouts, eingebetteter Tabellen, Formeln, mehrsheetiger Arbeitsmappen.
2. **Beantwortet vier Frageklassen:** `lookup`, `extraction`, `table-math`, `synthesis` — jede wird an eine andere LLM-Stufe geroutet.
3. **Zweisprachig DE + EN** erstklassig; Embedder und alle LLMs sind in beiden Sprachen stark.
4. **Jede Antwort zitiert.** Der System-Prompt erzwingt eine Ablehnung, wenn kein abgerufener Chunk eine Aussage stützt. Zitate verweisen auf Originaldatei, Seite und Abschnitt.
5. **ACL-bewusst.** Jede Qdrant-/DuckDB-Abfrage trägt einen verpflichtenden Gruppen-Filter; der Filter ist eine Zusicherung, kein Konfigurationsflag.
6. **Strukturbewusstes Retrieval.** Tabellen überleben die Ingestion als typisierte Spaltendaten in DuckDB *zusätzlich* zum Embedden. Tabellen-Mathematik nutzt echtes SQL.
7. **Hybrid-Retrieval.** Dense + sparse in einem einzigen Qdrant-Aufruf, durch RRF fusioniert, anschließend durch einen Cross-Encoder rerankt.
8. **Offline-Vertrag.** Keine ausgehenden Verbindungen zur Laufzeit — in Phase 9 mit pfSense / Little Snitch verifiziert.
9. **Vollständiger Trace pro Anfrage** in Langfuse: `classify` → `rewrite` → `dense/sparse` → `rerank` → `SQL` → `generate`.

2.2 Was bewusst NICHT enthalten ist (v1)

- Kein Web-Browsing, keine E-Mail-/Slack-Ingestion, keine agentischen Workflows jenseits von optionalem n8n.
- Keine Hochverfügbarkeit oder Clusterung (Single-Node + Backups).
- Keine feingranularen ACLs pro Dokument (nur auf Ordner-Ebene).

- Keine anderen Sprachen außer DE + EN in der Evaluationsmatrix.
- Keine Datei-Anhänge im Chat (würden ACLs umgehen).

Siehe [docs/01_CONCEPT.md](#) §10.

3. Technologie-Stack

Alles ist unter freizügigen Lizenzen (Apache 2.0 / MIT / PostgreSQL / BSD-3) verfügbar.

Ebene	Auswahl	Version / Tag	Begründung (ADR)
Dokument-Parsing	Docling (IBM)	3 GB Python-Image	Best-in-class für Tabellen + Layout; Apache 2.0 (ADR-001)
OCR	EasyOCR (Standard) / Tesseract deu+eng	im Image	Per Env austauschbar
Embeddings (dense + sparse)	bge-m3	ollama pull bge-m3 , 1024-d	Ein Modell, zwei Modi, DE+EN (ADR-003)
Vektor-DB	Qdrant	qdrant/qdrant:v1.12.0	Natives Hybrid + Payload-Filter (ADR-002)
Reranker	BAAI/bge-reranker-v2-m3 über TEI	cpu-1.5 (Standard)	Mehrsprachiger Cross-Encoder, auf Metal schnell
LLM-Server	Ollama	ollama/ollama:0.3.12	Apple-Silicon-nativ, lazy load, OpenAI-kompatibel
LLMs (gestuft)	Gemma 2 9B / Qwen 2.5 32B / Llama 3.3 70B	Q5/Q4 Quantisierungen	Routing nach Frageklasse (ADR-004)
XLSX / Tabellen	DuckDB	eingebettet, dateibasiert	SQL-Pfad für numerische Fragen (ADR-006)
Objektspeicher	MinIO	RELEASE.2024-08-03T...	Unveränderliche Rohdateien
Metadaten	PostgreSQL 16	postgres:16-alpine	Benutzer-Spiegel, Ordner, Dokumente, Chunks, Audit, Jobs
Warteschlange	Redis + RQ	redis:7-alpine	Einfache Ingestions-Queue
Identität	Authentik	2024.8	OIDC + Gruppen + Blueprints (ADR-005)
UI	Open WebUI + Pipelines	main	OIDC-fähig, eigene Pipeline brückt zur retrieval-api (ADR-007)
Observability (LLM)	Langfuse (self-hosted)	langfuse/langfuse:2.71	Span-pro-Schritt-Tracing
Observability (Infrastruktur)	Prometheus v2.55.0 , Grafana 11.2.0 , Loki 2.9.10	JSON-Dashboards, provisioniert	Container-/Host-Metriken, Logs
Reverse-Proxy	Caddy	caddy:2-alpine	Auto-HTTPS über interne CA
Orchestrierung (optional, Profil automation)	n8n	1.60.0	Geplante Jobs, Ordner-Watcher

Nicht als Framework übernommen (siehe [ADR-008](#)): LangChain / LlamaIndex / Haystack. Deren Chunker und SQL-Hilfen können als Komponenten hinter Feature-Flags importiert werden; der Kleber ist expliziter FastAPI-Code.

4. Architektur auf einen Blick

4.1 Komponenten-Inventar (25 Container im maximalen Profil)

Kategorie	Services
Edge	caddy
Identität	authentik-server, authentik-worker, authentik-db, authentik-redis
Speicher	postgres, redis, minio, qdrant, duckdb-api (+ eingebettete DuckDB-Datei)
LLM-Runtime	ollama, ollama-init (Einmal-Init), tei-reranker
Eigene Services	docling, ingestion-api, ingestion-worker, retrieval-api, duckdb-api
UI	openwebui, pipelines
Observability	langfuse, langfuse-db, prometheus, grafana, loki, promtail
Automatisierung (Profil)	n8n, watcher

Host-seitig offen: **nur 80 und 443** (über Caddy). Alles andere läuft über das interne Docker-Bridge-Netz `reineke` per Service-Name.

4.2 Ingestion-Datenfluss

```
Datei-Drop → ingestion-api → MinIO (raw/{doc_id}/{filename})
                                   → Postgres rag.documents (status=queued)
                                   → Redis-Queue (RQ)

Worker ← Redis
        → Docling /parse (DoclingDocument + Tabellen)
        → HybridChunker (strukturbewusst, max. 512 Tokens)
        → Ollama embed (bge-m3 dense, 1024-d)
        → bge-m3 sparse (in-process)
        → Qdrant upsert (dense + sparse + Payload inkl. acl_groups)
        → DuckDB-Tabellen (XLSX-Sheets, zuverlässige PDF-Tabellen)
        → Postgres rag.chunks / rag.tables
        → Postgres rag.documents (status=indexed)
```

Idempotent: (folder_path, sha256) dedupliziert; Fehler sind transaktional (keine verwaisten Punkte).

4.3 Retrieval- und Generierungs-Fluss

Frage in Open WebUI → Pipelines → retrieval-api (JWT durchgereicht)

retrieval-api:

1. verify_jwt (JWKS gecached)
2. principal.groups → ACL-Filter
3. classify(query) → lookup | extraction | table-math | synthesis [Gemma 9B]
4. (optional) HyDE / Paraphrase x2
5. embed(q) dense + sparse
6. Qdrant prefetch:
 - dense (Top 50, Filter acl_groups ANY groups)
 - sparse (Top 50, derselbe Filter)
 - Fusion per RRF
 - Top 50
7. TEI rerank → Top 12
8. falls table-math: LLM schreibt SQL → duckdb-api validiert + führt aus gegen
views.v_<table>_<group_hash> (ACL im SQL verankert)
9. Prompt bauen (DE/EN/zweisprachig), LLM-Stufe wählen
10. Ollama stream → SSE-Tokens
11. SSE-Event „citations“ (doc_id, chunk_id, page, scores, preview)
12. Postgres rag.audit_log + Langfuse-Trace

Anti-Halluzinations-Disziplin: enthält kein reranker Chunk im Top-K eine Stützung, antwortet das System in der Sprache der Nutzerin mit einer expliziten „nicht gefunden“-Zeile. Es gibt keinen „Kreativ-Modus“-Schalter.

4.4 Speicher-Layout

```
${DATA_ROOT:-/var/lib/reineke}/
├─ postgres/           Haupt-App-DB (rag-Schema)
├─ authentik-db/       Identity-DB
├─ langfuse-db/        Tracing-DB
├─ redis/              AOF
├─ minio/              raw/{doc_id}/... + export/
├─ qdrant/             Vektoren + Snapshots
├─ duckdb/             reineke.duckdb (einzelne Datei)
├─ ollama/             Modell-Gewichte (~80 GB bei allen Stufen)
├─ tei/               Reranker-Cache
├─ docling/           OCR-Modell-Cache
├─ loki/              Logs
└─ grafana/           Dashboards + State
```

Alle Dashboards sind provisionierte JSON-Dateien; kein Admin-Klick nötig, um sie bereitzustellen. Alle Mounts liegen als Unterverzeichnisse pro Service unter einem gemeinsamen Wurzelpfad — das hält Backup-/Restore-Skripte lesbar.

5. Datenmodell (PostgreSQL — Schema rag)

Sieben Tabellen bilden alle Belange ab, die weder Vektor noch Blob sind:

- `rag.users` — Spiegel von Authentik sub/email/groups, bei jeder JWT-Validierung aktualisiert.
- `rag.folders` — logischer Ordnerbaum; `acl_groups TEXT[]` ist verbindlich.

- `rag.documents` — jede eingeleseene Datei (inkl. `sha256`, `status`, `minio_key`, `pages`).
- `rag.chunks` — Admin-Sichtbarkeits-Spiegel; Vektoren leben in Qdrant.
- `rag.tables` — eine Zeile pro DuckDB-registrierter Tabelle (XLSX-Sheet oder PDF-Tabelle).
- `rag.audit_log` — jede retrieval-api-Anfrage: Benutzer, Klasse, abgerufene `doc_ids`, SQL (falls erzeugt), LLM, Tokens, Latenz, Hash der Antwort, Langfuse-Trace-Referenz.
- `rag.jobs` — RQ-Spiegel für die Admin-Ansicht (ephemerer Zustand wird persistent).

Die DDL findet sich in [docs/02_ARCHITECTURE.md §4](#).

6. Sicherheits- und ACL-Modell

Vier Säulen:

1. **Identität:** Authentik (OIDC, RS256 / 2048 Bit, Access-Token 15 Min, Refresh 24 h). Gruppen sind die Autorisierungseinheit.
2. **Vertrauensgrenze:** Caddy ist die einzige host-exponierte Oberfläche. Service-zu-Service nutzt entweder Bearer-JWT (Authentik) oder einen gemeinsamen `INTERNAL_SERVICE_TOKEN`.
3. **ACL-Durchsetzung:** `acl_groups` ist ein indiziertes Payload-Feld auf jedem Qdrant-Punkt. Jede Suche trägt einen verpflichtenden Filter `payload.acl_groups ANY user.groups`. DuckDB stellt pro Gruppen-Hash eigene Views bereit; die `duckdb-api` parst das generierte SQL, lehnt alles außer `SELECT` ab und führt ausschließlich gegen den ACL-View aus.
4. **Audit:** jede Anfrage (inkl. Antwort-Hash) landet in `rag.audit_log`. Authentik-Login-Ereignisse gehen in Loki. GDPR-Export per `rag-admin audit export --format csv`.

Notausgang: ein langlebiges, zufällig rotiertes `ADMIN_BACKUP_TOKEN` gewährt Admin-Zugang, falls Authentik selbst ausfällt. Der Einsatz wird deutlich protokolliert.

7. LLM-Routing (ADR-004)

Eine YAML-Router-Konfiguration wird von `retrieval-api` geladen:

```
classes:
  lookup: { model: gemma2:9b-instruct-q5_K_M, max_tokens: 400 }
  extraction: { model: qwen2.5:32b-instruct-q4_K_M, max_tokens: 1200 }
  table-math: { model: qwen2.5:32b-instruct-q4_K_M, max_tokens: 800 }
  synthesis: { model: llama3.3:70b-instruct-q4_K_M, max_tokens: 1600 }
  embedding: { model: bge-m3, dimensions: 1024 }
  reranker: { model: BAAI/bge-reranker-v2-m3, server: tei }
```

Ressourcen-Profil auf der M4-Max-Referenzmaschine (64 GB):

Modell	Quant	RAM ca.	Rolle	Ø Wartezeit bis zum ersten Token
Gemma 2 9B	Q5_K_M	7,5 GB	schneller Pfad, Klassifizierer	≤ 2 s
Qwen 2.5 32B	Q4_K_M	20 GB	Reasoning, Extraktion, Tabellen- Mathematik	≤ 6 s
Llama 3.3 70B	Q4_K_M	40 GB	Synthese (selten geladen)	≤ 20 s
bge-m3	–	1 GB	dense + sparse Embedding	–
bge-reranker-v2-m3	–	0,6 GB	Reranker (TEI)	p95 ≤ 500 ms / 12 Kandidaten

OLLAMA_MAX_LOADED_MODELS=2 auf Apple Silicon verhindert, dass Metal thrashed. Mit LLM_PROFILE=compact entfällt die schwerste Stufe komplett.

8. Observability

Drei getrennte Signale, bewusst getrennt:

- **Langfuse** — ein Eltern-Span pro Anfrage mit Kindern für `classify`, `rewrite`, `dense_search`, `sparse_search`, `rerank`, `sql_plan`, `sql_exec`, `generate`. Modellname, Eingaben, Ausgaben, Latenz, Token-Zählungen werden angeheftet. Die primäre Debug-Oberfläche.
- **Prometheus + Grafana** — `rag_query_total{class}`, `rag_query_latency_seconds{phase,class}`, `rag_retrieval_hits{source}`, `rag_ingestion_jobs_total{state}` sowie Container/Host-Metriken. Vier provisionierte Dashboards: Overview / Ingestion / Infra / Quality.
- **Loki + Promtail** — Standard-Output jedes Containers. Retention: 14 Tage INFO, 90 Tage WARN+.

Jeder eigene Service liefert `/metrics` (Prometheus-Format) und `/healthz` sowie einen `rag_build_info{version,commit}`-Gauge.

9. Build-Modell: Koordinator + sieben Spezialisten

Das Repository ist darauf ausgelegt, **von KI-Agenten** gebaut zu werden, nicht manuell. Acht Agent-Definitionen liegen in `.claude/agents/`, jede mit klarer Spur:

Agent	Phase(n)	Eigentumsbereich (im Repo)
coordinator	alle	BUILD_LOG.md , HANDOVER.md ; delegiert, verifiziert, schreibt nie Anwendungscode
deployment-agent	1, 9	docker-compose.yml , Makefile , .env.example , Caddyfile, Backup-/Restore-Skripte
auth-agent	2	Authentik-Blueprints, services/common/auth.py (JWT-Bibliothek)
llm-agent	3	scripts/pull-models.sh , scripts/smoke-*.sh , config/retrieval/models.yaml
ingestion-agent	4, 5	services/docling/** , services/ingestion-api/** , services/duckdb-api/** , migrations/**
retrieval-agent	6, 8	services/retrieval-api/** , config/retrieval/prompts/** , scripts/eval.py , Regressions-Tests
ui-agent	7	config/openwebui/** , config/pipelines/reineke_rag.py
observability-agent	9	config/langfuse/** , config/prometheus/** , config/grafana/provisioning/** , config/loki/**

Jede Phase hat **explizite Abnahmekriterien** (A1.1 bis A10.1), die der Koordinator durch Skripts verifiziert — nie durch Vertrauen auf Selbstreports eines Sub-Agenten. Bei Misserfolg wird der zuständige Sub-Agent mit Log-Evidenz erneut beauftragt. Drei Fehlschläge eskalieren zum menschlichen Eigentümer.

Siehe [docs/05_IMPLEMENTATION_PLAN.md](#) für die Phasen-Gates und [docs/06_AGENT_BRIEFS.md](#) für die Verträge pro Agent.

Grober Aufwand: ca. 18 Ingenieurstage äquivalente Arbeit; 3–5 Wall-Clock-Tage für einen Agenten-Build bei responsivem Eigentümer in den Phase-0- und Phase-8-Schleifen.

10. Architecture Decision Records

Acht akzeptierte ADRs halten die wesentlichen Entscheidungen fest:

#	Titel	Kern-Ergebnis
ADR-001	Docling für Dokument-Parsing	Ein Parser, Struktur erhalten, HybridChunker als Default
ADR-002	Qdrant für Vektor + Sparse	Natives Hybrid (1.10+), indizierte Payload-Filter, int8-Quant
ADR-003	bge-m3 als einziger Embedder	Dense + Sparse aus einem Modell, 1024-d, DE+EN gleichwertig
ADR-004	Gestufter Ollama-Stack + Router	9B / 32B / 70B pro Frageklasse geroutet
ADR-005	Authentik als IdP	OIDC + Gruppen, per Blueprint reproduzierbar
ADR-006	DuckDB-SQL-Pfad	Numerische Antworten werden <i>berechnet</i> , nicht halluziniert
ADR-007	Open WebUI + eigene Pipeline	Nur „Reineke-RAG“ als Modell sichtbar; Chat-Upload deaktiviert
ADR-008	Schlanke FastAPI-Services statt LangChain/LlamaIndex	Komponenten ja, Frameworks nein

Daumenregel: steht etwas weder in einem ADR noch in der Architektur, ist es Implementierungsdetail und jeder Sub-Agent entscheidet innerhalb seiner Spur frei.

11. Warum das n8n Self-Hosted-AI-Starterkit verworfen wurde

Das n8n-Kit ist eine brauchbare *Demo*. Reineke-RAG existiert, weil es an realen Office-Korpora aus vier strukturellen Gründen bricht ([Konzept §1](#)):

1. **Flatter Text-Parser** zerstört Tabellenstruktur in PDFs und XLSX.
2. **Fix-Größen-Chunking** trennt mitten in Tabellen und reißt Überschriften von Fließtext.
3. **Dense-only-Retrieval** verfehlt exakte Begriffe (Artikelnummern, Produktcodes, deutsche Komposita).
4. **Ein LLM für alle Fragen** — kein Pfad, der eine Summe in einer Tabelle tatsächlich berechnet, kein synthese-geeignetes Modell für dokumentübergreifende Arbeit.

Reineke-RAG kehrt jeden dieser Punkte um: strukturbewusster Parser (Docling) → strukturbewusstes Chunking (HybridChunker) → Hybrid-Retrieval + Rerank (Qdrant + TEI) → gestufter LLM-Routing mit echtem SQL-Zweig für Tabellen.

Zusätzlich bringt es mit, was dem Starter-Kit fehlt: **Zitate**, **Ordner-ACLs**, **Audit-Log**, **Offline-Vertrag**, **zweisprachige Prompts**, **Observability pro Anfrage**.

12. Operative Kennzahlen

Dimension	Ziel
Unterstützter Korpus	500 – 10 000 Dokumente (wachsend)
Ingestion-Durchsatz	100 gemischte Dokumente < 20 min auf M4 Max
Retrieval-Qualitätsgate	Recall@3 ≥ 85 %, Recall@10 ≥ 95 % auf 50 Gold-Queries
Zitier-Treue	100 % (kein halluziniertes Zitat erlaubt)
lookup -Latenz	p95 ≤ 5 s bis zum letzten Token
synthesis -Latenz	p95 ≤ 60 s
Qdrant-Dimensionierung (1 Mio. Chunks, int8)	~10 GB Disk, ~4 GB RAM
Ollama-Modell-Obergrenze	~80 GB (alle Stufen + Embed + Rerank)
Backup-Takt	nächtlich; GFS 7 täglich / 4 wöchentlich / 12 monatlich
Admin-Einarbeitung ausschließlich per Handbuch	< 30 min

13. Vorwärtskompatibilität

Erweiterungspunkte sind dokumentiert, damit v1.x-Ergänzungen keinen Umbau erzwingen:

- **Neue Dateiformate** → Parser-Zweig im Docling-Service hinzufügen (PPTX, HTML, MD nahezu kostenlos).
- **Neuer Embedder** → neue Qdrant-Collection (Vektor-Form ändert sich), Blue/Green-Reindex.
- **Neue ACL-Prädikate** (pro-Dokument-Tags, Vertraulichkeitsstufe) → Payload-Felder + Filterklauseln ergänzen; Migration füllt nach.
- **Neue Datenquellen** (Wiki-Crawl, E-Mail) → auf ein DoclingDocument-ähnliches JSON normalisieren; an `ingestion-api` übergeben.
- **Zwei-Host-Split** (v1.1) sobald Korpus > 5 000 Dokumente und Team > 30 aktive Nutzer — dokumentierter Pfad: Worker-Host (Ollama + TEI + ingestion-worker) + App-Host (der Rest) über Docker-Overlay oder WireGuard.

Die Rückwärtskompatibilität ist in den Versionsregeln explizit: ein Embedder-Wechsel oder eine Chunking-Änderung, die Grenzen verschiebt, ist ein breaking MAJOR-Bump; Prompt-Änderungen und LLM-Wechsel sind MINOR/PATCH.

14. Zusammenfassung

Reineke-RAG ist ein **bewusst langweiliger** RAG-Stack — langweilig im anerkennenden, Dan-McKinley-Sinn. Jede heiße Komponente hat irgendwo in Produktion ihren bewährten Vertreter: Postgres, Redis, MinIO, Qdrant, Docker, OIDC. Die *interessanten* Entscheidungen konzentrieren sich dort, wo sie messbare Qualität kaufen: Docling für strukturerhaltendes Parsen, bge-m3 für dense+sparse aus einem Modell, gestuftes LLM-Routing, ein echter SQL-Pfad für Tabellen, ein gemeinsamer Reranker. Alles liegt um einen einzigen Offline-Vertrag und einen verpflichtenden ACL-Filter.

Das Repository ist heute die Blaupause. Der Stack wird durch einen Agenten-Build instanziiert, den das Repository selbst orchestriert.

Zum aktuellen Zustand des Repositories, in dieser Reihenfolge lesen:

1. [README.md](#) — Orientierung
2. [docs/01_CONCEPT.md](#) — das „Warum“
3. [docs/02_ARCHITECTURE.md](#) — das „Wie“
4. [docs/adr/](#) — das „Warum nicht X“
5. [docs/05_IMPLEMENTATION_PLAN.md](#) — der Bauplan

Den fertigen Stack nutzen → siehe [USER_HANDBOOK_DE.md](#). **Den fertigen Stack betreiben** → siehe [TECHNICAL_HANDBOOK_DE.md](#).